Lab 02 · Build y pruebas automatizadas con AWS CodeBuild

Duración estimada: 55 minutos **Sesión:** 1 **Entorno:** AWS CloudShell · región `eu-west-1` (Irlanda)

Objetivo

Crear un proyecto de CodeBuild que, a partir de un código fuente en S3, ejecute análisis de estilo y pruebas unitarias, publique el informe de pruebas en la consola, lea un parámetro de Parameter Store y genere un artefacto. Después romperás una prueba para ver cómo falla el build y activarás la caché.

Requisitos

- Permisos sobre CloudFormation, IAM (crear roles), S3, CodeBuild, CloudWatch Logs y Systems Manager Parameter Store en la cuenta de formación.

Punto de partida

La práctica es independiente. En el paso 1 despliegas con CloudFormation un bucket y el rol de servicio que usará CodeBuild.

Paso 1 · Desplegar el punto de partida

```
export SUFIJO=jlb01            # tu sufijo, en minúsculas
export AWS_REGION=eu-west-1
mkdir -p ~/lab02 && cd ~/lab02

cat > inicio-lab02.yaml <<'EOF'
AWSTemplateFormatVersion: "2010-09-09"
Description: Lab 02 - punto de partida (bucket y rol de CodeBuild)
Parameters:
  Sufijo:
    Type: String
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub "lab02-${Sufijo}-${AWS::AccountId}"
      VersioningConfiguration: { Status: Enabled }
  RolCodeBuild:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub "lab02-codebuild-${Sufijo}"
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
            Principal: { Service: codebuild.amazonaws.com }
            Action: sts:AssumeRole
      Policies:
        - PolicyName: permisos-build
          PolicyDocument:
            Version: "2012-10-17"
            Statement:
              - Effect: Allow
                Action: [logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents]
                Resource: "*"
              - Effect: Allow
                Action: [s3:GetObject, s3:GetObjectVersion, s3:PutObject, s3:GetBucketLocation,
s3:ListBucket]
                Resource: [!GetAtt Bucket.Arn, !Sub "${Bucket.Arn}/*"]
              - Effect: Allow
                Action: [codebuild:CreateReportGroup, codebuild:CreateReport, codebuild:UpdateReport,
                         codebuild:BatchPutTestCases, codebuild:BatchPutCodeCoverages]
                Resource: !Sub "arn:aws:codebuild:${AWS::Region}:${AWS::AccountId}:report-group/*"
```

```
              - Effect: Allow
                Action: [ssm:GetParameters, ssm:GetParameter]
                Resource: !Sub
"arn:aws:ssm:${AWS::Region}:${AWS::AccountId}:parameter/formacion/${Sufijo}/*"
Outputs:
  Bucket: { Value: !Ref Bucket }
  Rol: { Value: !GetAtt RolCodeBuild.Arn }
EOF

aws cloudformation deploy --stack-name lab02-inicio-$SUFIJO \
  --template-file inicio-lab02.yaml --parameter-overrides Sufijo=$SUFIJO \
  --capabilities CAPABILITY_NAMED_IAM

export BUCKET=$(aws cloudformation describe-stacks --stack-name lab02-inicio-$SUFIJO \
  --query "Stacks[0].Outputs[?OutputKey=='Bucket'].OutputValue" --output text)
echo $BUCKET
```

Crea además un parámetro que el build leerá (simula un dato de configuración):

```
aws ssm put-parameter --name /formacion/$SUFIJO/entorno --value "formacion" --type String --overwrite
```

Paso 2 · Aplicación, pruebas y buildspec

```
mkdir -p app/src app/tests && cd app

cat > src/calculadora.py <<'EOF'
def precio_con_iva(base: float, tipo: float = 0.21) -> float:
    if base < 0:
        raise ValueError("La base no puede ser negativa")
    return round(base * (1 + tipo), 2)


def descuento(precio: float, porcentaje: float) -> float:
    if not 0 <= porcentaje <= 100:
        raise ValueError("Porcentaje fuera de rango")
    return round(precio * (1 - porcentaje / 100), 2)
EOF
touch src/__init__.py

cat > tests/test_calculadora.py <<'EOF'
import pytest
from src.calculadora import precio_con_iva, descuento


def test_iva_general():
    assert precio_con_iva(100) == 121.0


def test_iva_reducido():
    assert precio_con_iva(100, 0.10) == 110.0


def test_base_negativa():
    with pytest.raises(ValueError):
        precio_con_iva(-1)


def test_descuento():
    assert descuento(200, 25) == 150.0
EOF

cat > requirements-dev.txt <<'EOF'
pytest==8.3.3
ruff==0.6.9
EOF

cat > buildspec.yml <<'EOF'
version: 0.2
env:
  parameter-store:
    ENTORNO: /formacion/SUFIJO_AQUI/entorno
phases:
  install:
```

```
    runtime-versions:
      python: 3.12
    commands:
      - pip install -r requirements-dev.txt
  pre_build:
    commands:
      - echo "Build para el entorno $ENTORNO"
      - ruff check src tests
  build:
    commands:
      - python -m pytest --junitxml=reports/junit.xml
  post_build:
    commands:
      - echo "Empaquetando version $CODEBUILD_RESOLVED_SOURCE_VERSION"
      - zip -r calculadora.zip src
reports:
  pruebas-calculadora:
    files: [junit.xml]
    base-directory: reports
    file-format: JUNITXML
artifacts:
  files: [calculadora.zip]
EOF
sed -i "s/SUFIJO_AQUI/$SUFIJO/" buildspec.yml
```

Comprueba que las pruebas pasan en local antes de subir nada (buena práctica: la CI no es tu primera línea de pruebas):

```
pip install --user -q -r requirements-dev.txt && python3 -m pytest -q
```

Empaqueta el código fuente y súbelo al bucket:

```
zip -r ../fuente.zip . -x "*.pyc" -x "__pycache__/*" -x ".pytest_cache/*"
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
```

Paso 3 · Crear el proyecto de CodeBuild

En la consola: **CodeBuild → Build projects → Create build project**.

Sección	Valor
Project name	`lab02-${SUFIJO}` (sustituye por tu sufijo)
Source provider	Amazon S3
Bucket	tu bucket `lab02-<sufijo>-<cuenta>`
S3 object key	`fuente/fuente.zip`
Provisioning model	On-demand
Environment image	Managed image
Compute	EC2 · Operating system: Amazon Linux · Runtime: Standard · Image: la más reciente (`aws/codebuild/amazonlinux-x86_64-standard:5.0` o superior)
Service role	Existing service role → `lab02-codebuild-<sufijo>`
Buildspec	Use a buildspec file
Artifacts	Amazon S3 · bucket: el tuyo · Name: `artefactos` · Artifacts packaging: None
Logs	CloudWatch logs activado (valores por defecto)

Desmarca la casilla que permite a CodeBuild modificar el rol (ya tiene los permisos necesarios) y pulsa **Create build project**.

> Si prefieres la línea de comandos, el equivalente es `aws codebuild create-project` con esos mismos valores; en la formación usamos la consola para ver todas las opciones.

Paso 4 · Primer build y revisión

Pulsa **Start build**. Mientras se ejecuta:

1. En la pestaña **Build logs** sigue la salida en tiempo real. Localiza la línea `Build para el entorno formacion`: el valor viene de Parameter Store.
2. En **Phase details** observa cuánto duró cada fase. ¿Cuál es la más lenta?
3. Al terminar, abre **Reports** (en el menú de CodeBuild, *Report groups*) y entra en `lab02-<sufijo>-pruebas-calculadora`: verás los cuatro casos de prueba superados.
4. Comprueba el artefacto:

```
aws s3 ls s3://$BUCKET/artefactos/ --recursive
```

> **Punto de control 1.** Build en `Succeeded`, informe con 4/4 pruebas y `calculadora.zip` en el bucket.

Paso 5 · Romper una prueba

Introduce un error en la lógica y vuelve a subir el código:

```
cd app
sed -i 's/return round(base \* (1 + tipo), 2)/return round(base * tipo, 2)/' src/calculadora.py
zip -r ../fuente.zip . -x "*.pyc" -x "__pycache__/*" -x ".pytest_cache/*"
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
aws codebuild start-build --project-name lab02-$SUFIJO --query "build.id" --output text
```

Sigue el build en la consola.

> **Punto de control 2.** El build falla en la fase `BUILD`, el informe muestra dos pruebas fallidas con su mensaje de error, y **no** se genera un artefacto nuevo. Esa es la función de la CI: impedir que un cambio defectuoso avance.

Deshaz el error, vuelve a subir el código y lanza otro build para dejarlo en verde:

```
cd app
sed -i 's/return round(base \* tipo, 2)/return round(base * (1 + tipo), 2)/' src/calculadora.py
zip -r ../fuente.zip . -x "*.pyc" -x "__pycache__/*" -x ".pytest_cache/*"
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
aws codebuild start-build --project-name lab02-$SUFIJO --query "build.id" --output text
```

Paso 6 · Activar la caché y comparar

1. En el proyecto, **Edit → Artifacts → Additional configuration → Cache type: Local** y marca **Custom cache**. Guarda.
2. Añade al final del `buildspec.yml`:

```
cache:
  paths:
    - "/root/.cache/pip/**/*"
```

3. Sube el código y lanza dos builds seguidos. Compara la duración de la fase `INSTALL` entre ambos.

> La caché local solo se reutiliza si el siguiente build cae en el mismo host poco después; para una caché persistente usa el tipo **Amazon S3**. Con dependencias tan ligeras la diferencia será pequeña, pero en proyectos reales puede ahorrar minutos por build.

Reto opcional

Añade a la fase `build` la medición de cobertura con `pytest-cov`, genera un informe en formato Cobertura (`--cov=src --cov-report=xml:reports/coverage.xml`) y publícalo como segundo informe en la sección `reports` con `file-format: COBERTURAXML`.

Preguntas de reflexión

1. ¿Por qué ejecutamos `ruff` en `pre_build` y no después de las pruebas?
2. ¿Qué ventaja tiene leer `ENTORNO` de Parameter Store frente a definirlo como variable en el proyecto?

Limpieza

```
aws codebuild delete-project --name lab02-$SUFIJO
for rg in $(aws codebuild list-report-groups --query "reportGroups[?contains(@, 'lab02-$SUFIJO')]" --output
text); do
  aws codebuild delete-report-group --arn $rg --delete-reports; done
aws ssm delete-parameter --name /formacion/$SUFIJO/entorno
aws s3 rm s3://$BUCKET --recursive
aws s3api delete-objects --bucket $BUCKET --delete "$(aws s3api list-object-versions --bucket $BUCKET \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' --output json)" 2>/dev/null
aws s3api delete-objects --bucket $BUCKET --delete "$(aws s3api list-object-versions --bucket $BUCKET \
  --query '{Objects: DeleteMarkers[].{Key:Key,VersionId:VersionId}}' --output json)" 2>/dev/null
aws cloudformation delete-stack --stack-name lab02-inicio-$SUFIJO
aws logs delete-log-group --log-group-name /aws/codebuild/lab02-$SUFIJO 2>/dev/null
```